Two Pointers Level 3 Problems

1. Container With Most Water
Pattern: Two Pointers + Greedy Movement

```
def max_area(height):

    left = 0
    right = len(height) - 1

    max_area = 0

    while left < right:

        width = right - left

        current_height = min(
            height[left],
            height[right]
        )

        area = width * current_height

        max_area = max(max_area, area)

        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_area


height = [1,8,6,2,5,4,8,3,7]
```

2. 3Sum
Pattern: Sorting + Two Pointers

```
def sort_colors(nums):

    low = 0
    mid = 0
    high = len(nums) - 1

    while mid <= high:

        if nums[mid] == 0:

            nums[low], nums[mid] = nums[mid], nums[low]

            low += 1
            mid += 1

        elif nums[mid] == 1:

            mid += 1

        else:  # nums[mid] == 2

            nums[mid], nums[high] = nums[high], nums[mid]

            high -= 1

nums = [2,0,2,1,1,0]
```

3. Sort Colors
Pattern: Three Pointers

```
def sort_colors(nums):

    low = 0
    mid = 0
    high = len(nums) - 1

    while mid <= high:

        if nums[mid] == 0:

            nums[low], nums[mid] = nums[mid], nums[low]

            low += 1
            mid += 1

        elif nums[mid] == 1:

            mid += 1

        else:  # nums[mid] == 2

            nums[mid], nums[high] = nums[high], nums[mid]

            high -= 1

nums = [2,0,2,1,1,0]
```

4. Valid Palindrome II
Pattern: Two Pointers + Conditional Skip

```
def valid_palindrome(s):

    def is_palindrome(left, right):

        while left < right:
            if s[left] != s[right]:
                return False

            left += 1
            right -= 1
        return True

    left = 0
    right = len(s) - 1

    while left < right:

        if s[left] != s[right]:

            return (
                is_palindrome(left + 1, right)
                or
                is_palindrome(left, right - 1)
            )

        left += 1
        right -= 1

    return True

s = "abca"
```